INTENDENCIA DE SAN JOSÉ

PORTAL VIAL

Administración de Sistemas Operativos

Integrantes

Tomás Cabrera · Juan Robaina · Gabriela Romero · Verónica Romero · Juan Corrales

INTENDENCIA DE SAN JOSÉ

PORTAL VIAL

Análisis de requerimientos

Integrantes

Tomás Cabrera · Juan Robaina · Gabriela Romero · Verónica Romero · Juan Corrales

Análisis de requerimientos de infraestructura

Proyecto: Educación Vial IMSJ - Portal Vial

Plataforma: Ubuntu Server 26.04 LTS

1. Objetivo

Definir la infraestructura necesaria para ejecutar el portal público, el panel IMSJ, la API PHP y la base MySQL en un servidor Linux administrable, recuperable y accesible desde la red autorizada.

2. Carga de trabajo

Dato	Valor utilizado
Usuarios simultáneos	90
Aplicaciones web	1
Bases de datos	1
Contenedores auxiliares	0
Perfil	Pequeña producción
Datos iniciales	1 GB estimado
Crecimiento mensual	10 % estimado
Proyección	12 meses
Backups completos	1 por día
Retención local en la VM base	2 días
Retención externa inicial	7 copias diarias
Administración	Cockpit
Umbral de revisión	70 % sostenido de CPU, RAM o disco
Copia externa	Sí
RAID1	No, por tratarse de una VM con backup externo

La estimación de 90 usuarios simultáneos contempla una situación de difusión concentrada, por ejemplo compartir el portal mediante un código QR durante una charla en un liceo. Dos clases pueden aportar aproximadamente 60 accesos al mismo tiempo; el resto de la capacidad permite atender a ciudadanos y personal que ya estén utilizando el sitio. Este valor describe sesiones con el portal abierto, no 90 solicitudes PHP ejecutándose de forma permanente.

El portal entrega principalmente HTML, CSS, JavaScript e imágenes estáticas. PHP y MySQL intervienen en solicitudes breves de consulta, autenticación, reserva y administración. Por esa razón, el dimensionamiento se basa en el tipo y la duración de las operaciones, los dos contenedores desplegados y un margen operativo, en lugar de asignar una cantidad fija de memoria por usuario. La concurrencia y el crecimiento deberán contrastarse con métricas reales.

3. Dimensionamiento

Se adopta una configuración inicial que cubre Ubuntu Server, Docker, Cockpit, Apache/PHP y MySQL, y que puede ampliarse después de medir el entorno objetivo:

Recurso	Fundamento	Configuración inicial
CPU	Operaciones HTTP y SQL breves, sin procesamiento intensivo	**2 vCPU**
Memoria	SO, Docker, Apache/PHP, MySQL, Cockpit y margen para picos	**4 GB RAM**
Disco útil	SO, imágenes Docker, aplicación, datos, adjuntos, logs y las dos copias diarias más recientes	**30 GB SSD/NVMe**
Disco con 7 copias completas locales	Mayor retención dentro de la misma VM	**40-50 GB SSD/NVMe**

Los videos se conservan como enlaces externos y no ocupan almacenamiento local. Los elementos de mayor tamaño son los PDF y las imágenes cargados por el personal. La configuración de 30 GB requiere trasladar cada respaldo válido a un medio externo y limitar la retención dentro de la VM a dos días. Si se mantienen siete copias completas en la VM, se deben asignar entre 40 y 50 GB.

La configuración es un punto de partida, no una capacidad garantizada. Una prueba de carga de 90 sesiones deberá registrar solicitudes por segundo, latencia, errores, CPU, memoria e I/O. Si un recurso supera de forma sostenida el 70 %, se ajustará la VM.

4. Requerimientos del servidor

Código	Requerimiento
RS-01	Utilizar Ubuntu Server 26.04 LTS de 64 bits.
RS-02	Asignar inicialmente 2 vCPU, 4 GB de RAM y 30 GB de disco SSD/NVMe.
RS-03	Configurar un usuario administrativo con `sudo`, sin trabajar habitualmente como `root`.
RS-04	Habilitar OpenSSH Server para la administración remota.
RS-05	Configurar una IP fija mediante Netplan.
RS-06	Aplicar UFW con denegación de entradas por defecto y permitir solo SSH, aplicación y Cockpit desde redes autorizadas.
RS-07	Instalar Docker Engine desde el repositorio oficial y Docker Compose v2.
RS-08	Ejecutar la aplicación y MySQL mediante `compose.yaml`.
RS-09	Mantener MySQL sin puerto publicado hacia el host.
RS-10	Conservar base, adjuntos y caché en volúmenes persistentes.
RS-11	Reiniciar los contenedores automáticamente después de reiniciar el servidor.
RS-12	Limitar el tamaño y la cantidad de archivos de log de Docker.
RS-13	Realizar un backup diario de base, adjuntos, código y configuración.
RS-14	Mantener localmente las dos copias diarias más recientes y al menos siete copias diarias en un medio externo protegido; usar 40-50 GB si se requieren siete copias completas dentro de la VM.
RS-15	Probar la restauración sobre una instalación aislada antes de considerar válido un backup.
RS-16	Supervisar CPU, RAM, disco, logs, contenedores y estado de los servicios.
RS-17	Validar la configuración con 90 sesiones simultáneas y ampliar recursos si CPU, RAM o disco superan de forma sostenida el 70 %.

5. Servicios y puertos

Servicio	Puerto	Exposición
SSH	22/TCP	Solo administración autorizada
Portal, panel y API	8000/TCP	Red autorizada
Cockpit	9090/TCP	Solo administración autorizada
MySQL	3306/TCP	Únicamente dentro de Docker

La demostración utiliza HTTP en una red controlada. Una publicación en Internet requiere dominio, HTTPS y un proxy inverso administrado por la institución.

6. Criterios de aceptación

- Ubuntu informa una versión 26.04 LTS actualizada.
- La IP del servidor permanece estable después de reiniciar.
- SSH, UFW, Docker y Cockpit se encuentran activos.
- app y db aparecen saludables en Docker Compose.
- El portal responde desde otra computadora autorizada de la red.
- MySQL no acepta conexiones directas desde la red.
- Los datos y adjuntos permanecen después de recrear los contenedores.
- El backup genera archivos verificables mediante SHA-256.
- Una restauración aislada recupera cuentas, contenidos, adjuntos, consultas y reservas.
- Una prueba de carga de 90 sesiones simultáneas confirma tiempos de respuesta y consumo de CPU, memoria e I/O aceptables antes de la publicación institucional.

Referencias

- Calculadora didáctica de requerimientos
- Servidor Linux con Docker
- Ciclo de soporte de Ubuntu
- Instalación oficial de Docker Engine en Ubuntu

INTENDENCIA DE SAN JOSÉ

PORTAL VIAL

Arquitectura propuesta

Integrantes

Tomás Cabrera · Juan Robaina · Gabriela Romero · Verónica Romero · Juan Corrales

Arquitectura propuesta de infraestructura

Proyecto: Educación Vial IMSJ - Portal Vial

Servidor: Ubuntu Server 26.04 LTS, 2 vCPU, 4 GB RAM y 30 GB SSD/NVMe

1. Vista general

MERMAID

```
flowchart TB
    Admin[Equipo administrador] -->|SSH 22| UFW[UFW]
    Admin -->|Cockpit 9090| UFW
    Usuarios[Usuarios autorizados] -->|HTTP 8000| UFW

    UFW --> Host[Ubuntu Server 26.04 LTS]
    Host --> Docker[Docker Engine y Compose v2]
    Host --> Cockpit[Cockpit]
    Host --> Backup[Backup Bash y cron]

    Docker --> App[app: Apache y PHP 8.5]
    App --> DB[db: MySQL 8.4]
    App --- Portal[frontend-publico]
    App --- Panel[frontend-imsj]

    DB --- DBData[(db_data)]
    App --- Uploads[(app_uploads)]
    App --- Cache[(cache)]

    Backup --> Local[(2 copias locales)]
    Local -. copia .-> Externo[(7 copias diarias protegidas)]
```

El servidor concentra la administración del sistema operativo y Docker. Los usuarios solo acceden a Apache; MySQL permanece en la red interna de Compose.

2. Servidor anfitrión

Elemento	Definición
Sistema operativo	Ubuntu Server 26.04 LTS de 64 bits
Recursos	2 vCPU, 4 GB RAM y 30 GB SSD/NVMe
Capacidad de referencia	Objetivo inicial de 90 sesiones simultáneas, sujeto a prueba de carga y ampliación si un recurso supera 70 % sostenido
Nombre sugerido	`srv-imsj`
Red	IP fija configurada mediante Netplan
Administración	OpenSSH y Cockpit
Firewall	UFW con entradas denegadas por defecto
Contenedores	Docker Engine y Docker Compose v2
Automatización	Bash y cron

3. Componentes Docker

Componente	Responsabilidad
`app`	Ejecutar Apache, PHP, la API y servir los dos frontends.
`db`	Mantener la base MySQL sin publicar el puerto 3306.
`db_data`	Conservar tablas, cuentas, contenidos, consultas, reservas e historial.
`app_uploads`	Conservar PDF e imágenes cargados.
`cache`	Conservar contadores compartidos de intentos de autenticación.

Los contenedores utilizan `restart: unless-stopped`, límites de logs y controles de salud. `app` se ejecuta como `www-data`; `db`, como `mysql`. Ambos usan raíz de solo lectura, `no-new-privileges` y capacidades reducidas.

4. Red y acceso

Origen	Destino	Puerto	Regla
Red administrativa	Ubuntu Server	22/TCP	SSH
Red administrativa	Ubuntu Server	9090/TCP	Cockpit
Red autorizada	Aplicación	8000/TCP	Portal, panel y API
app	db	3306/TCP	Red interna Docker

Compose publica la aplicación en `0.0.0.0:8000`. UFW determina qué redes pueden acceder. No se publica MySQL.

Para una demostración controlada se utiliza HTTP. Si el sistema se publica en Internet, se debe colocar un proxy inverso con HTTPS en 443/TCP, mantener la aplicación y MySQL detrás de él y reemplazar las URL de `.env` por el dominio institucional.

5. Recorrido de una petición

```mermaid
sequenceDiagram
    actor Usuario
    participant Frontend as Navegador / Frontend
    participant Apache
    participant Router as public/index.php
    participant Auth as Autenticación y validación
    participant Controller as Controlador
    participant Service as Servicio
    participant Repository as Repositorio PDO
    participant DB as MySQL

    Usuario->>Frontend: Ejecuta una acción
    Frontend->>Apache: Solicitud HTTP a /api
    Apache->>Router: Reescritura y entrada única
    Router->>Controller: Despacha método y ruta
    opt Ruta protegida
        Controller->>Auth: Verifica sesión, cuenta y rol
        Auth-->>Controller: Usuario autorizado o error HTTP
    end
    Controller->>Controller: Valida los datos de entrada
    Controller->>Service: Entrega datos validados
    Service->>Repository: Aplica reglas y solicita persistencia
    Repository->>DB: Consulta o transacción SQL preparada
    DB-->>Repository: Filas o resultado
    Repository-->>Service: Datos del dominio
    Service-->>Controller: Resultado del caso de uso
    Controller-->>Frontend: Respuesta JSON o 204
    Frontend-->>Usuario: Actualiza la interfaz
```

Los frontends consumen `/api` desde el mismo origen. Apache sirve los archivos públicos y dirige las rutas de API a `public/index.php`; el router identifica método y URL y selecciona el controlador. En las operaciones protegidas, el controlador solicita a `Auth` la verificación de sesión, cuenta y rol antes de modificar o exponer datos. El controlador valida la entrada específica, el servicio aplica reglas y transacciones, y el repositorio concentra las consultas PDO preparadas. La respuesta vuelve por esas capas hasta el navegador. Las rutas públicas omiten el paso de autenticación. El resto del backend permanece fuera de `DocumentRoot`.

6. Persistencia y recuperación

El backup diario reúne:

- dump comprimido de MySQL;
- adjuntos del volumen `app_uploads`;
- copia del proyecto;
- configuración necesaria para reconstruir la instalación;
- manifiesto, versión y hashes SHA-256.

La VM base conserva dos copias diarias locales y mantiene al menos siete copias diarias en un medio externo protegido. Si la política exige conservar siete backups completos dentro de la VM, el disco se amplía a 40-50 GB. La restauración se prueba con otro nombre de proyecto Compose y otro puerto para no tocar la instalación principal.

7. Monitoreo

Nivel	Herramientas
Host	Cockpit, `uptime`, `free -h`, `df -h`, `ss -tulpn`
Servicios	`systemctl status docker`, `journalctl -u docker`
Contenedores	`docker compose ps`, `docker stats`, `docker compose logs`
Aplicación	`/api/health` y control de salud interno de Compose
Backups	`SHA256SUMS`, manifiesto y prueba de restauración

8. Límites

- Los datos iniciales y su crecimiento son estimaciones y deben reemplazarse por mediciones reales.
- La capacidad de 90 usuarios simultáneos es un objetivo de dimensionamiento; debe confirmarse mediante una prueba de carga y observación en Cockpit antes de asumirla como capacidad operativa demostrada.
- Los 90 usuarios representan sesiones con períodos de lectura, no 90 solicitudes PHP activas permanentemente.
- Si CPU, memoria o disco superan de forma sostenida el 70 %, se deben ampliar los recursos o revisar la carga antes de la publicación.
- La instalación en red controlada no representa una publicación institucional con dominio y HTTPS.
- El backup no se considera válido hasta completar una restauración aislada.

Documentos relacionados

Análisis de requerimientos, justificación tecnológica, infraestructura y reconstrucción.

INTENDENCIA DE SAN JOSÉ

PORTAL VIAL

Justificación tecnológica

Integrantes

Tomás Cabrera · Juan Robaina · Gabriela Romero · Verónica Romero · Juan Corrales

Justificación tecnológica

Proyecto: Educación Vial IMSJ - Portal Vial

Decisiones de infraestructura

Tecnología	Justificación	Condición
Ubuntu Server 26.04 LTS	Es la versión LTS vigente, con mantenimiento estándar previsto hasta 2031 y soporte oficial para Docker Engine. Permite administración por terminal mediante Netplan, UFW y systemd.	Debe mantenerse actualizado y administrarse con un usuario `sudo`.
Docker Engine	Aísla la aplicación y la base del sistema anfitrión y permite reconstruir el entorno de forma repetible.	Se instala desde el repositorio oficial de Docker para Ubuntu.
Docker Compose v2	Define servicios, volúmenes, variables, dependencias y controles de salud en un solo archivo.	La configuración sensible permanece en `.env`, fuera de Git.
Apache y PHP 8.5	Ejecutan la API PHP y publican el portal y el panel desde el mismo origen.	Solo `public/` y los alias de frontend quedan expuestos.
MySQL 8.4	Conserva las relaciones, restricciones y transacciones del sistema.	El puerto 3306 permanece dentro de Docker.
Volúmenes Docker	Separan datos, adjuntos y caché del ciclo de vida de los contenedores.	Deben formar parte del backup y de la prueba de restauración.
Netplan	Mantiene una dirección estable para que los clientes localicen el servidor.	La interfaz, IP, gateway y DNS dependen de la red real.
UFW	Limita las conexiones entrantes a los servicios autorizados.	SSH debe permitirse antes de activar el firewall.
OpenSSH	Permite administrar el servidor sin interfaz gráfica.	Se utiliza un usuario administrativo distinto de `root`.
Cockpit	Permite observar servicios, recursos y registros desde una interfaz de administración.	El puerto 9090 se restringe a la red administrativa.
Bash y cron	Automatizan backups diarios y tareas operativas con herramientas nativas de Linux.	Los scripts generan hashes de integridad.

Dimensionamiento

La estimación considera 90 sesiones simultáneas, una aplicación PHP, una base MySQL, dos contenedores, Cockpit, archivos cargados por el personal y backups diarios. Una sesión abierta no equivale a una solicitud activa de forma permanente: los usuarios alternan solicitudes breves con períodos de lectura.

El portal sirve principalmente recursos estáticos. PHP y MySQL participan en consultas, autenticación, reservas y operaciones administrativas de corta duración, sin cálculos intensivos. Para esta carga se propone iniciar con **2 vCPU y 4 GB de RAM**, capacidad que incluye el sistema operativo, Docker, Apache/PHP, MySQL, Cockpit y margen para picos transitorios.

Se asignan **30 GB SSD/NVMe** para el sistema, las imágenes Docker, la aplicación, la base, los adjuntos, los logs y las dos copias diarias más recientes. Los videos se almacenan como enlaces externos. Las copias válidas se trasladan a un medio externo protegido; si deben conservarse siete backups completos dentro de la VM, el disco recomendado pasa a **40-50 GB**.

La configuración inicial es **2 vCPU, 4 GB de RAM y 30 GB SSD/NVMe**. Antes de afirmar su capacidad operativa se ejecutará una prueba con 90 sesiones y se medirán solicitudes por segundo, latencia, errores, CPU, memoria e I/O. Un consumo sostenido superior al 70 % activa la revisión o ampliación del recurso afectado.

Organización de los servicios

El despliegue mantiene dos contenedores:

- `app`: Apache, PHP, API y frontends;
- `db`: MySQL.

Esta separación permite actualizar la aplicación sin reemplazar los datos. MySQL no se expone hacia la red y solo `app` publica el puerto de la aplicación.

No se incorpora Portainer porque no es necesario para operar dos servicios y su acceso al socket de Docker amplía privilegios. Cockpit y la terminal permiten realizar las tareas administrativas requeridas.

Disponibilidad y operación

`restart: unless-stopped` permite recuperar los servicios después de reiniciar Ubuntu. Los healthchecks verifican aplicación y base; los logs se rotan para evitar crecimiento indefinido. La persistencia depende de los volúmenes, no de los contenedores.

Backup y recuperación

El backup incluye código, dump MySQL, adjuntos, configuración protegida, manifiesto y hashes. Se ejecuta una vez por día; la VM base conserva dos días y el medio externo protegido mantiene al menos siete copias diarias. La restauración se prueba sobre una instalación aislada.

Copiar únicamente el repositorio no recupera los datos. Copiar únicamente un volumen MySQL mientras está escribiendo tampoco sustituye un dump lógico consistente.

Seguridad

- Administración mediante usuario `sudo`, no mediante uso habitual de `root`.
- UFW con denegación de entradas por defecto.
- MySQL sin puerto publicado.
- Contraseñas únicamente en `.env` protegido.
- Contenedores con usuarios de servicio, raíz de solo lectura y capacidades reducidas.
- HTTPS obligatorio antes de una publicación en Internet.

INTENDENCIA DE SAN JOSÉ

PORTAL VIAL

Documentación de infraestructura

Integrantes

Tomás Cabrera · Juan Robaina · Gabriela Romero · Verónica Romero · Juan Corrales

Documentación de infraestructura

Proyecto: Educación Vial IMSJ - Portal Vial

Entorno objetivo: Ubuntu Server 26.04 LTS

1. Inventario del servidor

Elemento	Configuración
CPU	2 vCPU iniciales
Memoria	4 GB RAM
Disco	30 GB SSD/NVMe; 40-50 GB con siete backups completos locales
Red	IP fija con Netplan
Administración	OpenSSH y Cockpit
Firewall	UFW
Contenedores	Docker Engine y Docker Compose v2
Automatización	Bash y cron

2. Archivos operativos

Archivo	Función
`backend/Dockerfile`	Construye la imagen Apache/PHP e instala las extensiones necesarias.
`backend/compose.yaml`	Define aplicación, base, volúmenes, reinicio, salud, logs y exposición de red.
`backend/apache.conf`	Configura la API y los alias de los frontends.
`backend/.env.example`	Plantilla sin secretos para crear `.env`.
`backend/database.sql`	Crea el esquema inicial cuando el volumen de MySQL está vacío.
`backend/scripts/migrar.php`	Aplica cambios incrementales de esquema sin borrar datos.
`backend/scripts/crear_admin.php`	Crea la primera cuenta de personal.
`backend/scripts/salud.php`	Comprueba aplicación, base y portal.
`backend/scripts/backup.sh`	Genera backup completo, manifiesto y hashes.
`backend/scripts/restore.sh`	Restaura base y adjuntos en una instalación confirmada.

3. Servicios Docker

Servicio	Imagen	Usuario	Exposición	Persistencia
`app`	Construida desde `php:8.5-apache`	`www-data`	`APP_BIND:APP_PORT` hacia 80	`app_uploads`, `cache`
`db`	`mysql:8.4`	`mysql`	Sin puerto publicado	`db_data`

Ambos servicios utilizan `restart: unless-stopped`, raíz de solo lectura, `no-new-privileges`, capacidades reducidas y rotación de logs. `app` espera que `db` esté saludable.

4. Volúmenes

Volumen	Contenido	Incluido en backup
`db_data`	Datos internos de MySQL	Mediante dump lógico comprimido
`app_uploads`	PDF e imágenes cargados	Sí, como archivo comprimido
`cache`	Contadores temporales de autenticación	No; puede reconstruirse

5. Variables de configuración

Variable	Uso
`APP_ENV`	`production` en el servidor.
`APP_BIND`	`0.0.0.0` para escuchar en las interfaces del servidor.
`APP_PORT`	Puerto publicado, 8000 por defecto.
`APP_URL`	URL completa con IP fija o dominio.
`FRONTEND_ORIGIN`	Origen permitido para el navegador.
`DB_DATABASE`	Nombre de la base.
`DB_USERNAME`	Usuario de aplicación.
`DB_PASSWORD`	Contraseña del usuario de aplicación.
`DB_ROOT_PASSWORD`	Contraseña administrativa de MySQL.
`INITIAL_ADMIN_*`	Datos temporales para crear la primera cuenta.

`.env` debe pertenecer al administrador y utilizar permisos `600`. No se guarda en Git.

6. Red y firewall

La IP se configura mediante Netplan con los datos reales de la red. UFW aplica:

- denegar conexiones entrantes por defecto;
- permitir salidas;
- permitir 22/TCP desde la red administrativa;
- permitir 8000/TCP desde la red autorizada;
- permitir 9090/TCP únicamente desde la red administrativa;
- no permitir 3306/TCP desde el host.

El puerto publicado por Compose escucha en `0.0.0.0`, pero la autorización efectiva queda en UFW.

7. Operación

```bash
cd /srv/imsj/Proyecto-IMSJ/backend
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs --tail 100 app db
```

Para detener sin borrar datos:

```bash
docker compose stop
```

No se utiliza `docker compose down -v` durante la operación normal porque elimina los volúmenes persistentes.

8. Monitoreo

```bash
hostnamectl
ip -br address
uptime
free -h
df -h
ss -tulpn
systemctl status docker
docker compose ps
docker stats --no-stream
docker system df
```

Cockpit complementa estas comprobaciones desde el puerto 9090 restringido a administración.

9. Backup

Ejecución manual:

```bash
cd /srv/imsj/Proyecto-IMSJ/backend
chmod +x scripts/backup.sh scripts/restore.sh
./scripts/backup.sh /srv/backups/imsj
```

Cada backup contiene:

```text
backup-AAAA-MM-DD_HH-MM-SS/
├── base.sql.gz
├── adjuntos.tar.gz
├── proyecto.tar.gz
├── config/
│   ├── .env
│   ├── compose.yaml
│   ├── Dockerfile
│   └── apache.conf
├── MANIFIESTO.txt
└── SHA256SUMS
```

La carpeta contiene configuración sensible y debe almacenarse con acceso restringido. La VM de 30 GB conserva dos días de copias locales; las copias válidas se trasladan a un medio externo protegido, donde se mantienen al menos siete copias diarias. Para conservar siete backups completos dentro de la propia VM se amplía el disco a 40-50 GB.

10. Restauración

La restauración se realiza sobre una instalación aislada y requiere confirmación explícita:

```bash
IMSJ_RESTORE_CONFIRM=YES ./scripts/restore.sh /srv/backups/imsj/backup-FECHA
```

Después se comprueban portal, ingreso, contenidos, adjuntos, consultas, reservas y conteos de la base.

Documentos relacionados

Análisis, arquitectura, justificación y reconstrucción.

INTENDENCIA DE SAN JOSÉ

PORTAL VIAL

Reconstrucción de la infraestructura

Integrantes

Tomás Cabrera · Juan Robaina · Gabriela Romero · Verónica Romero · Juan Corrales

Reconstrucción de la infraestructura

Proyecto: Educación Vial IMSJ - Portal Vial

Sistema objetivo: Ubuntu Server 26.04 LTS

1. Preparar el servidor

Crear una máquina virtual o servidor con:

- 2 vCPU;
- 4 GB de RAM;
- 30 GB de disco SSD/NVMe, o 40-50 GB si se conservarán siete backups completos dentro de la VM;
- adaptador de red con acceso desde los equipos autorizados;
- ISO de Ubuntu Server 26.04 LTS.

Durante la instalación seleccionar Ubuntu Server, habilitar OpenSSH Server, crear un usuario administrativo distinto de `root` y utilizar el nombre `srv-imsj` o uno equivalente.

2. Actualizar Ubuntu

```bash
sudo apt update
sudo apt full-upgrade -y
sudo timedatectl set-timezone America/Montevideo
sudo reboot
```

Después del reinicio:

```bash
hostnamectl
free -h
df -h
ip -br address
ip route
```

La salida debe identificar Ubuntu 26.04 LTS y los recursos asignados.

3. Configurar la IP fija

Identificar la interfaz y los datos entregados por la red:

```bash
ip -br address
ip route
```

Editar el archivo de Netplan disponible en `/etc/netplan/`. Ejemplo:

```yaml
network:
  version: 2
  ethernets:
    INTERFAZ:
      dhcp4: false
      addresses:
        - IP_SERVIDOR/PREFIJO
      routes:
        - to: default
          via: PUERTA_DE_ENLACE
      nameservers:
        addresses:
          - DNS_1
          - DNS_2
```

Validar antes de aplicar, especialmente si la sesión es remota:

```bash
sudo netplan try
sudo netplan apply
ip -br address
ip route
```

Los valores de interfaz, IP, prefijo, gateway y DNS deben corresponder a la red real; no se copian literalmente los nombres del ejemplo.

4. Configurar SSH y UFW

Instalar las herramientas necesarias:

```bash
sudo apt install -y openssh-server ufw curl ca-certificates git jq gzip
sudo systemctl enable --now ssh
```

Permitir SSH antes de activar el firewall. Sustituir `RED_ADMIN` y `RED_USUARIOS` por las redes reales, por ejemplo una red con formato `192.168.1.0/24`.

```bash
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow from RED_ADMIN to any port 22 proto tcp
sudo ufw allow from RED_USUARIOS to any port 8000 proto tcp
sudo ufw allow from RED_ADMIN to any port 9090 proto tcp
sudo ufw enable
sudo ufw status verbose
```

No se abre el puerto 3306 de MySQL.

5. Instalar Docker Engine y Compose

Eliminar paquetes incompatibles si estuvieran instalados:

```bash
for pkg in docker.io docker-doc docker-compose podman-docker containerd runc; do
  sudo apt remove -y "$pkg"
done
```

Agregar el repositorio oficial:

```bash
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

sudo tee /etc/apt/sources.list.d/docker.sources >/dev/null <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
sudo systemctl enable --now docker
sudo docker run --rm hello-world
docker compose version
```

Agregar el usuario administrativo al grupo Docker y volver a iniciar sesión:

```bash
sudo usermod -aG docker "$USER"
```

Pertenecer al grupo Docker otorga privilegios administrativos sobre el servidor; no se agrega a usuarios comunes.

6. Instalar Cockpit

```bash
sudo apt install -y cockpit
sudo systemctl enable --now cockpit.socket
systemctl is-active cockpit.socket
```

Cockpit queda disponible en `https://IP_SERVIDOR:9090` únicamente desde la red administrativa permitida en UFW.

7. Copiar el proyecto

Crear la ubicación de trabajo y copiar o clonar la versión de entrega:

```bash
sudo mkdir -p /srv/imsj
sudo chown "$USER":"$USER" /srv/imsj
cd /srv/imsj
git clone URL_DEL_REPOSITORIO Proyecto-IMSJ
cd Proyecto-IMSJ/backend
```

Si el proyecto se entrega mediante pendrive, copiar la carpeta completa a `/srv/imsj/Proyecto-IMSJ` conservando ambos frontends y el backend.

8. Configurar el entorno

```bash
cp .env.example .env
nano .env
```

Completar como mínimo:

```dotenv
APP_ENV=production
APP_BIND=0.0.0.0
APP_PORT=8000
APP_URL=http://IP_SERVIDOR:8000
FRONTEND_ORIGIN=http://IP_SERVIDOR:8000
DB_DATABASE=imsj
DB_USERNAME=imsj
DB_PASSWORD=CLAVE_SEGURA
DB_ROOT_PASSWORD=OTRA_CLAVE_SEGURA
```

Aplicar permisos:

```bash
chmod 600 .env
docker compose config --quiet
```

Las claves reales no se guardan en Git ni se muestran en capturas.

9. Iniciar el sistema

```bash
docker compose up -d --build
docker compose ps
docker compose logs --tail 100 app db
```

Comprobar desde el servidor:

```bash
curl -I http://127.0.0.1:8000/
curl http://127.0.0.1:8000/api/health
```

Desde otra computadora autorizada abrir:

```text
http://IP_SERVIDOR:8000/frontend-publico/
http://IP_SERVIDOR:8000/frontend-imsj/
```

10. Crear la primera cuenta

```bash
read -r -p "Cédula: " INITIAL_ADMIN_CEDULA
read -r -p "Nombre: " INITIAL_ADMIN_NOMBRE
read -r -s -p "Contraseña: " INITIAL_ADMIN_PASSWORD
echo
export INITIAL_ADMIN_CEDULA INITIAL_ADMIN_NOMBRE INITIAL_ADMIN_PASSWORD

docker compose exec \
  -e INITIAL_ADMIN_CEDULA \
  -e INITIAL_ADMIN_NOMBRE \
  -e INITIAL_ADMIN_PASSWORD \
  app php scripts/crear_admin.php

unset INITIAL_ADMIN_CEDULA INITIAL_ADMIN_NOMBRE INITIAL_ADMIN_PASSWORD
```

Las cuentas siguientes se crean desde el panel.

11. Configurar backups diarios

```bash
sudo mkdir -p /srv/backups/imsj
sudo chown "$USER":"$USER" /srv/backups/imsj
cd /srv/imsj/Proyecto-IMSJ/backend
chmod +x scripts/backup.sh scripts/restore.sh
./scripts/backup.sh /srv/backups/imsj
```

Verificar los hashes del backup generado:

```bash
cd /srv/backups/imsj/backup-FECHA
sha256sum -c SHA256SUMS
```

Programar una ejecución diaria a las 02:00:

```bash
crontab -e
```

```cron
0 2 * * * cd /srv/imsj/Proyecto-IMSJ/backend && BACKUP_RETENTION_DAYS=2 ./scripts/backup.sh /srv/backups/imsj >> /srv/backups/imsj/backup.log 2>&1
```

La retención de dos días corresponde a la VM base de 30 GB. Copiar cada backup válido a un medio externo protegido y conservar allí al menos siete copias diarias. Si se desea usar la retención local predeterminada de siete días del script, asignar 40-50 GB de disco. La copia externa no debe quedar montada permanentemente con permisos de escritura para todos los usuarios.

12. Probar la restauración

La prueba se realiza en otra carpeta y con otro nombre de proyecto Compose para evitar los volúmenes principales.

```bash
cp -a /srv/imsj/Proyecto-IMSJ /srv/imsj/Proyecto-IMSJ-restauracion
cd /srv/imsj/Proyecto-IMSJ-restauracion/backend
```

Editar `.env` y utilizar `APP_PORT=8001`, `APP_URL=http://IP_SERVIDOR:8001` y `FRONTEND_ORIGIN=http://IP_SERVIDOR:8001`. Permitir temporalmente el puerto 8001 solo desde la red administrativa.

```bash
export COMPOSE_PROJECT_NAME=imsj_restauracion
IMSJ_RESTORE_CONFIRM=YES ./scripts/restore.sh /srv/backups/imsj/backup-FECHA
docker compose ps
curl http://127.0.0.1:8001/api/health
```

Comprobar cuentas, contenidos, adjuntos, consultas y reservas. Registrar fecha, backup, hash, versión, resultado y responsable. Al terminar, detener el entorno de prueba sin tocar el principal.

13. Verificación después de reiniciar

```bash
sudo reboot
```

Después del reinicio:

```bash
systemctl is-active docker
docker compose -f /srv/imsj/Proyecto-IMSJ/backend/compose.yaml \
  --env-file /srv/imsj/Proyecto-IMSJ/backend/.env ps
curl http://127.0.0.1:8000/api/health
sudo ufw status verbose
```

Los contenedores deben iniciar automáticamente por su política `restart: unless-stopped`.